Intro to Multiple Linear Regression

মাল্টিপল লিনিয়ার রিগ্রেশন কী?

সিম্পল লিনিয়ার রিগ্রেশন কেন সিম্পল বলা হয়?

এটি রিগ্রেশন প্রবলেম সলভ করে।
এটি লিনিয়ার ইকুয়েশন ব্যবহার করে।
এতে মাত্র একটি ভেরিয়েবল (ফিচার) থাকে।

মাল্টিপল লিনিয়ার রিগ্রেশন কেন মাল্টিপল বলা হয়?

কারণ এতে একের অধিক ফিচার থাকে।
মানে ইনপুট হিসেবে মাল্টিপল ভেরিয়েবল ব্যবহার করে প্রেডিকশন করা হয়।

সিম্পল আর মাল্টিপলের মূল পার্থক্য

বৈশিষ্ট্য	সিম্পল লিনিয়ার রিগ্রেশন	মাল্টিপল লিনিয়ার রিগ্রেশন
ফিচারের সংখ্যা	১টি	একাধিক (২ বা তার বেশি)
ইকুয়েশন ফর্ম	$y = m_1x_1 + b$	$y = m_1x_1 + m_2x_2 + ... + b$
উদাহরণ	শুধু বছর অফ এক্সপেরিয়েন্স দিয়ে স্যালারি প্রেডিকশন	এক্সপেরিয়েন্স + পিএস স্কিল + পস টাইম + জিপিএ দিয়ে স্যালারি প্রেডিকশন

ডেটাসেট নিয়ে বিস্তারিত ব্যাখ্যা

ধরা যাক, একটি কোম্পানি হায়ারিং করার সময় শুধু বছর অফ এক্সপেরিয়েন্স দেখে না। তারা আরও কিছু বিষয় দেখে:

নতুন যুক্ত ফিচারগুলো:

পিএস স্কিল — প্রবলেম সলভিং ক্যাপাবিলিটি (স্কোর 1 থেকে 10 বা 100 পর্যন্ত হতে পারে)
পস টাইম — ইন্টারভিউতে কত সময় পজিটিভ না হয়ে নেগেটিভ ভাব দেখিয়েছে (যত কম ভালো)
জিপিএ — সার্টিফিকেটে উল্লেখিত জিপিএ
বছর অফ এক্সপেরিয়েন্স — আগের মতোই আছে

টার্গেট কলাম: স্যালারি (যা প্রেডিক্ট করতে চাই)

ডেটা পয়েন্ট ও ফিচার সংখ্যা চিহ্নিতকরণ

প্রতীক ব্যবহার:

m = ডেটা পয়েন্টের সংখ্যা (কতগুলো রো আছে)
n = ফিচারের সংখ্যা (ইনপুট কলাম কয়টি)

এই ডেটাসেটে:

মোট ডেটা পয়েন্ট = ৫টি (একটি টেবিলের ৫টি সারি)
ফিচার সংখ্যা = ৪টি (পিএস স্কিল, পস টাইম, জিপিএ, এক্সপেরিয়েন্স)
টার্গেট কলাম = ১টি (স্যালারি)

সুতরাং:

```
m = 5
n = 4
```

ম্যাথমেটিক্যাল উপস্থাপনা

আগে ছিল ওয়ান-ডি অ্যারে (শুধু এক্সপেরিয়েন্স)

এখন লাগবে টু-ডি অ্যারে (ম্যাট্রিক্স)

X (ইনপুট): একটি ম্যাট্রিক্স যেখানে m সংখ্যক রো (ডেটা পয়েন্ট) এবং n সংখ্যক কলাম (ফিচার)
প্রতি রোতে = n সংখ্যক ভ্যালু
y (টার্গেট): একটি ভেক্টর, যেখানে m সংখ্যক ভ্যালু (প্রতি ডেটা পয়েন্টের জন্য একটি স্যালারি)

উদাহরণ:

```
X = [
  [ps_skill1, pos_time1, gpa1, exp1],
  [ps_skill2, pos_time2, gpa2, exp2],
  ...
]
y = [salary1, salary2, salary3, ...]
```

সমীকরণের ফর্ম

মাল্টিপল লিনিয়ার রিগ্রেশনের ইকুয়েশন:

$$\hat{y} = b_0 + b_1x_1 + b_2x_2 + b_3x_3 + ... + b x$$

এখানে:

$\hat{y}$ = প্রেডিক্টেড স্যালারি
b_0 = ইন্টারসেপ্ট (যেখানে সব ফিচার শূন্য)
b_1, b_2... = প্রতিটি ফিচারের জন্য কোএফিশিয়েন্ট বা ওয়েট
x_1, x_2... = ফিচারের মান

উদাহরণস্বরূপ আমাদের ডেটাসেটের জন্য:

$$\text{স্যালারি} = b_0 + b_1 \times \text{PS_skill} + b_2 \times \text{pos_time} + b_3 \times \text{gpa} + b_4 \times \text{experience}$$

কেন মাল্টিপল লিনিয়ার রিগ্রেশন গুরুত্বপূর্ণ?

বাস্তব জীবনের সমস্যাগুলোতে সাধারণত একাধিক ফিচার থাকে।
একাধিক ফিচার ব্যবহার করলে প্রেডিকশন অধিক নির্ভুল হয়।
যেমন: স্যালারি শুধু এক্সপেরিয়েন্সের ওপর নির্ভর করে না — স্কিল, জিপিএ, ইন্টারভিউ পারফরম্যান্স-ও গুরুত্বপূর্ণ।

Feature and Target Analysis

কেন আমরা স্ক্যাটার প্লট দেখি?

উদ্দেশ্য:

প্রত্যেকটি ফিচার কলামের সাথে টার্গেট কলামের (স্যালারি) কী ধরনের সম্পর্ক আছে — তা বোঝা।

কীভাবে করব?

প্রত্যেক ফিচারের জন্য আলাদা স্ক্যাটার প্লট আঁকি।

যেমন:

- এক্সপেরিয়েন্স vs স্যালারি
- পিএস স্কিল vs স্যালারি
- পস টাইম vs স্যালারি
- জিপিএ vs স্যালারি

দেখি ফিচারের মান বাড়লে স্যালারি বাড়ে কি কমে, নাকি কোনো সম্পর্ক নেই।

কোডে কীভাবে করব

ধাপ ১: ডেটা থেকে নির্দিষ্ট কলাম বের করা (স্লাইসিং)

আমাদের ডেটা এখন **2D** অ্যারে (ম্যাট্রিক্স)।

প্রতি সারি = একটি ডেটা পয়েন্ট।

প্রতি কলাম = একটি ফিচার।

স্লাইসিং সিনট্যাক্স:

```
1X[সব_সারি, কলামের_শুরু_ইন্ডেক্স : কলামের_শেষ_ইন্ডেক্স]
```

মনে রাখবেন: শেষ ইন্ডেক্সটি বাদ যায়। অর্থাৎ start : end মানে start থেকে end-1 পর্যন্ত।

ধাপ ২: ইয়ার্স অফ এক্সপেরিয়েন্স বের করে স্ক্যাটার প্লট দেখা

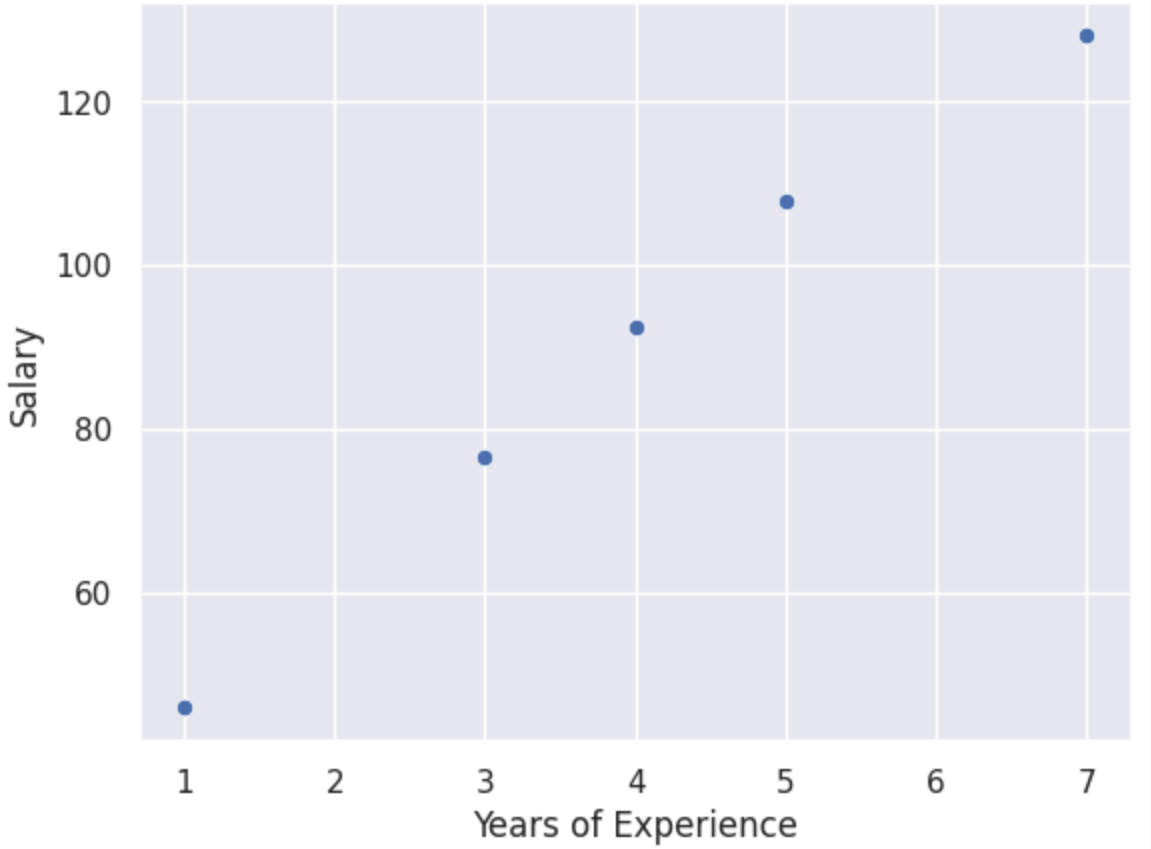
এক্সপেরিয়েন্স কলামের ইন্ডেক্স → **0** (শুরুর কলাম)

আমরা চাই শুধু এই একটি কলাম → 0:1 দিলে 0 নম্বর কলাম পাই।

```
1experience = X[:, 0:1].ravel()   # 2D থেকে 1D তে রূপান্তর
```

.ravel() কেন লাগে?

স্ক্যাটার প্লটের জন্য 1D ডেটা দরকার। 2D দিলে error আসতে পারে।



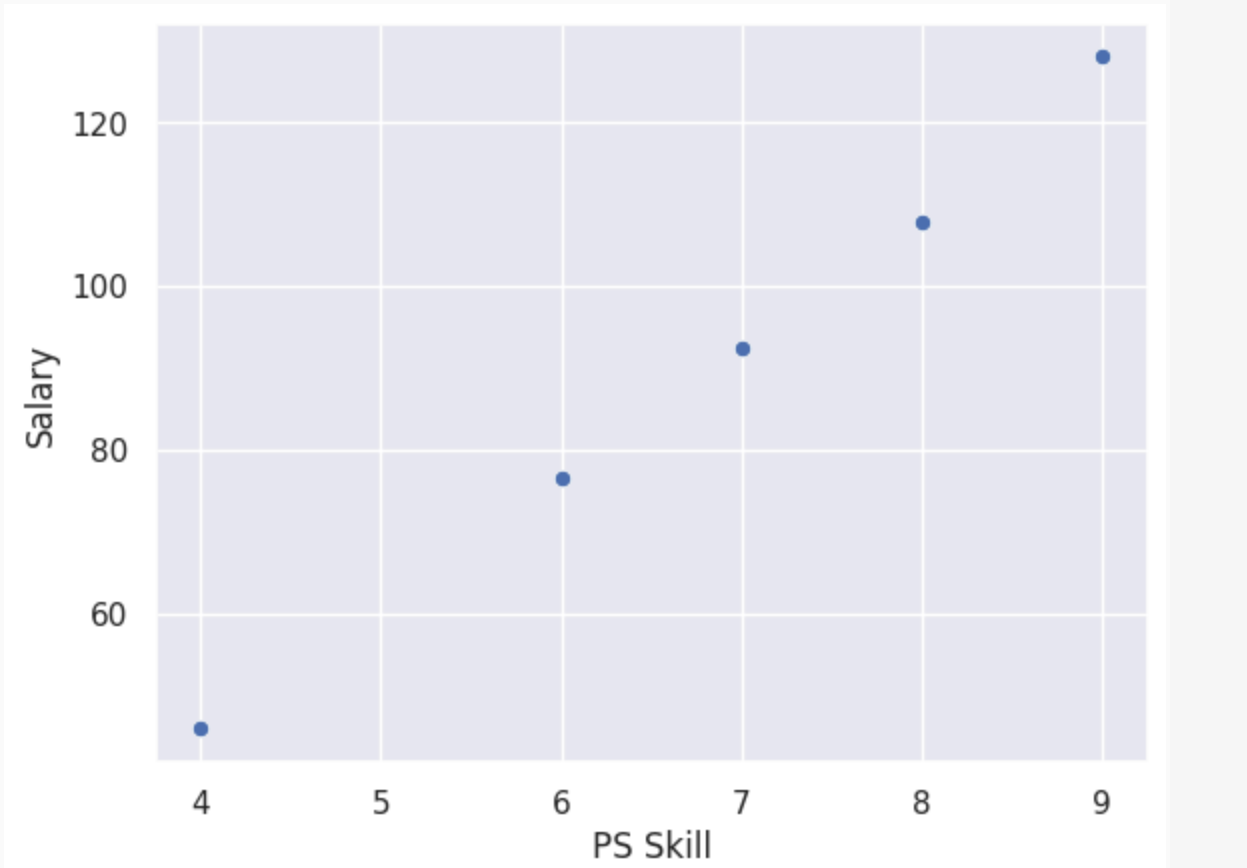
প্লট করার পর পর্যবেক্ষণ:
ইয়ার্স অফ এক্সপেরিয়েন্স বাড়লে স্যালারি বাড়ে → এটা পজিটিভ কোরিলেশন।

ধাপ ৩: পিএস স্কিল দেখা

পিএস স্কিল কলামের ইন্ডেক্স → **1**

স্লাইসিং: X[:, 1:2]

```
1ps_skill = X[:, 1:2].ravel()
```

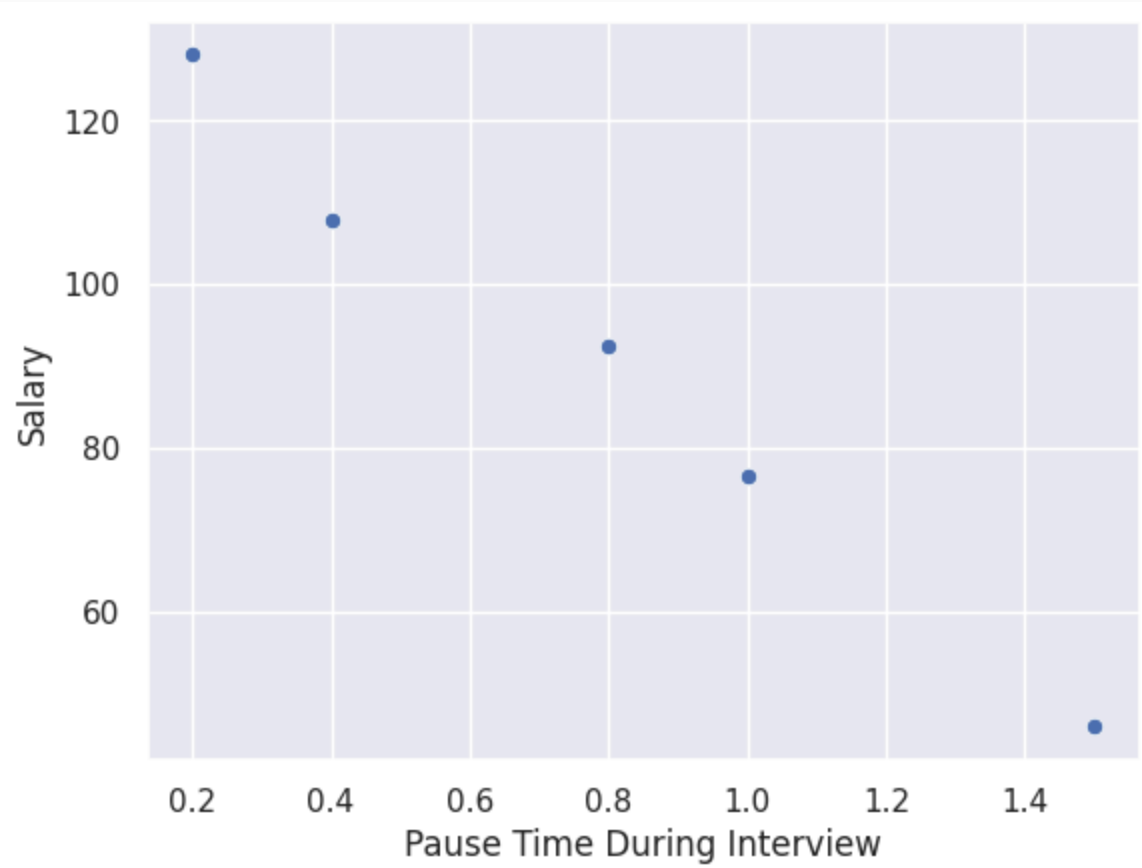


প্লট করে দেখি: এটাও পজিটিভ কোরিলেশন।
স্কিল বাড়লে স্যালারি বাড়ে।

ধাপ ৪: পস টাইম দেখা

পস টাইম কলামের ইন্ডেক্স → **2**
স্লাইসিং: X[:, 2:3]

```
1pos_time = X[:, 2:3].ravel()
```



প্লট করলে দেখা যায়:

পস টাইম বাড়লে স্যালারি কমে → এটা নেগেটিভ কোরিলেশন।

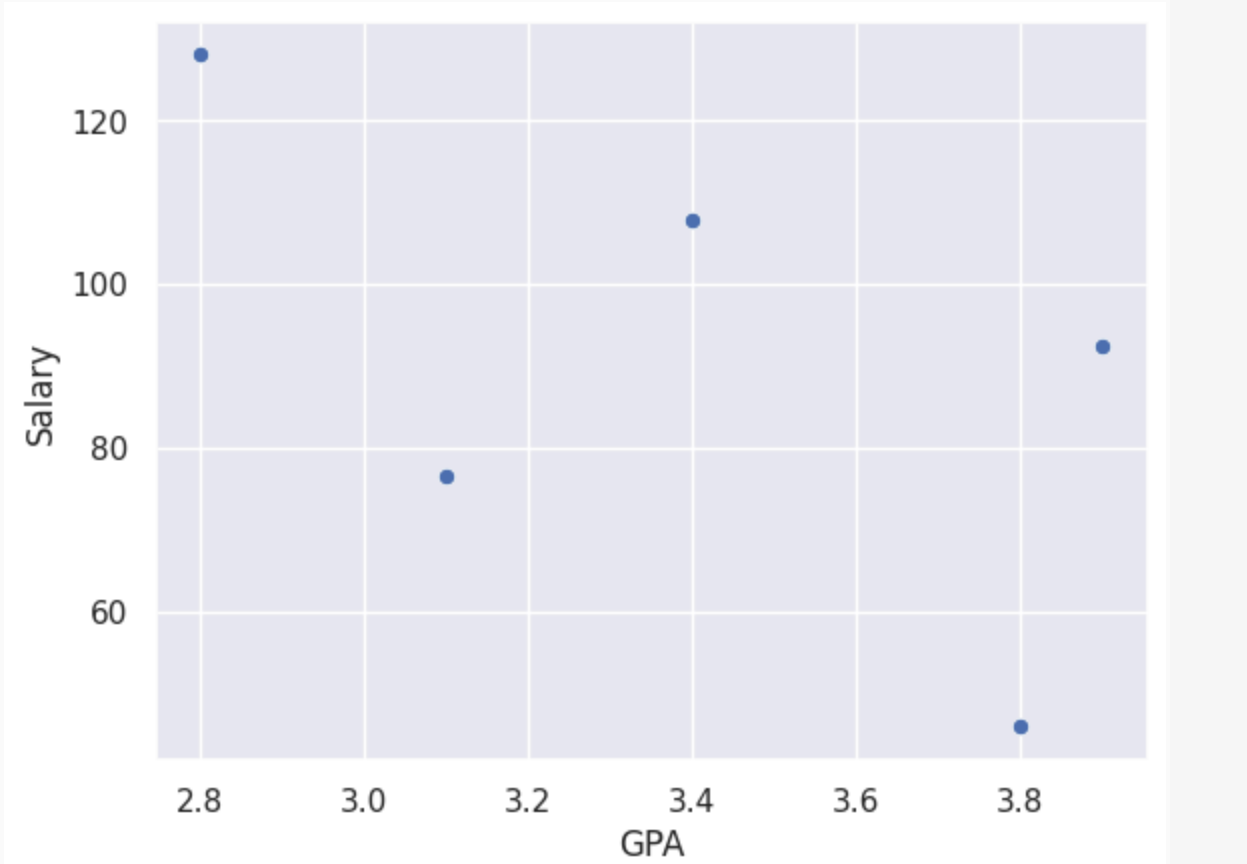
পস টাইম = ইন্টারভিউতে কতটা উত্তর দিতে পারেনি (বা নেতিবাচক মনোভাব) → যত বেশি, স্যালারি তত কম।

ধাপ ৫: জিপিএ দেখা

জিপিএ কলামের ইন্ডেক্স → **3** (শেষ কলাম)

স্লাইসিং: X[:, 3:4]

```
1gpa = X[:, 3:4].ravel()
```



প্লট করলে দেখা যায়:
জিপিএ বাড়ছে কমছে, কিন্তু স্যালারির তেমন কোনো নির্দিষ্ট সম্পর্ক নেই।
কখনো বেশি জিপিএতে কম স্যালারি, কখনো কম জিপিএতে বেশি স্যালারি।
→ এটা কোনো কোরিলেশন নেই (প্রায় শূন্য সম্পর্ক)।

কোরিলেশনের প্রকারভেদ

সম্পর্কের ধরন	কী হয়?	উদাহরণ
পজিটিভ কোরিলেশন	ফিচার বাড়লে টার্গেট বাড়ে	এক্সপেরিয়েন্স ↑ → স্যালারি ↑
নেগেটিভ কোরিলেশন	ফিচার বাড়লে টার্গেট কমে	পস টাইম ↑ → স্যালারি ↓
কোনো কোরিলেশন নেই	ফিচারের পরিবর্তনে টার্গেট এলোমেলো	জিপিএ আর স্যালারির সম্পর্ক নেই

Generalizing the Model

মডেল কী এবং কেন দরকার?

মডেল **=** একটি ম্যাথমেটিক্যাল ফাংশন

যাতে ইনপুট দিলে (এক্সপেরিয়েন্স, স্কিল, পস টাইম, জিপিএ) -> আউটপুট হিসেবে প্রেডিক্টেড স্যালারি পাওয়া যায়।

উদাহরণ**:**

আমি মডেলকে চারটি ইনপুট দেব:

ইয়ার্স অফ এক্সপেরিয়েন্স
পিএস স্কিল
পস টাইম
জিপিএ

মডেল আউটপুট দেবে → **ŷ (**প্রেডিক্টেড স্যালারি**)**

🎯 লক্ষ্য**:** এমন একটি ফাংশন বানানো, যা যেকোনো নতুন ডেটার জন্য সঠিক স্যালারি প্রেডিক্ট করতে পারে।

সিম্পল লিনিয়ার রিগ্রেশনের কথাও মনে পড়ে নাও

সিম্পল লিনিয়ার রিগ্রেশনের ইকুয়েশন ছিল:

$$\hat{y} = w \times x + b$$

এখানে:

w = ওয়েট (weight)
b = বায়াস (bias)
x = একমাত্র ফিচার (যেমন এক্সপেরিয়েন্স)

মাল্টিপল লিনিয়ার রিগ্রেশনে এই ফর্মুলা বড় হয় — কারণ এখন ফিচার একাধিক।

মাল্টিপল লিনিয়ার রিগ্রেশনের ইকুয়েশন

যদি ২টি ফিচার থাকে (x_1, x_2):

$$\hat{y} = w_1 \times x_1 + w_2 \times x_2 + b$$

যদি ৩টি ফিচার থাকে (x_1, x_2, x_3):

$$\hat{y} = w_1 \times x_1 + w_2 \times x_2 + w_3 \times x_3 + b$$

আমাদের ডেটাসেটে ৪টি ফিচার আছে (x_1, x_2, x_3, x_4):

$$\hat{y} = w_1 \times x_1 + w_2 \times x_2 + w_3 \times x_3 + w_4 \times x_4 + b$$

এখানে:

x_1 = ইয়ার্স অফ এক্সপেরিয়েন্স
x_2 = পিএস স্কিল
x_3 = পস টাইম
x_4 = জিপিএ
$\hat{y}$ = প্রেডিক্টেড স্যালারি

জেনারাল ফর্মুলা (n সংখ্যক ফিচারের জন্য)

যদি **n** টি ফিচার থাকে:

$$\hat{y} = w_1x_1 + w_2x_2 + w_3x_3 + \ldots + w_nx_n + b$$

নিয়ম:

- যত ফিচার, তত ওয়েট
- সব ওয়েটের গুণফল যোগ করে শেষে একটি বায়াস যোগ হয়

ভেক্টর ডট প্রোডাক্ট দিয়ে সহজ উপস্থাপন

আমরা ডট প্রোডাক্ট ব্যবহার করে পুরো হিসেব এক লাইনে লিখতে পারি।

ধাপ ১: ওয়েট ভেক্টর তৈরি করি

$$W = [w_1, w_2, w_3, w_4]$$

ধাপ ২: ফিচার ভেক্টর তৈরি করি

$$X = [x_1, x_2, x_3, x_4]$$

ধাপ ৩: ডট প্রোডাক্ট

ডট প্রোডাক্ট মানে:

প্রতিটি ওয়েটের সাথে সংশ্লিষ্ট ফিচার গুণ করে সব যোগ করা।

$$W \cdot X = w_1 \times x_1 + w_2 \times x_2 + w_3 \times x_3 + w_4 \times x_4$$

ধাপ ৪: পুরো মডেল

$$\hat{y} = (W \cdot X) + b$$

অর্থাৎ:

প্রেডিকশন **=** ডট প্রোডাক্ট**(**ওয়েট**,** ফিচার**) +** বায়াস

🧠 মনে রাখবেন**:**

$\hat{y} = W \cdot X + b$ → এই একটি লাইনই পুরো মাল্টিপল লিনিয়ার রিগ্রেশন।

মডেলের প্যারামিটার কী কী?

- **W (**ওয়েট ভেক্টর**):** প্রতিটি ফিচারের গুরুত্ব নির্ধারণ করে।

b (বায়াস): বেসিক ভ্যালু (যখন সব ফিচার শূন্য, তখন স্যালারি কত হবে)।

মডেল লার্নিং মানে:

ডেটা থেকে সঠিক **W** আর **b** বের করা — যাতে প্রেডিকশন সবচেয়ে নির্ভুল হয়।

Numpy operations over Normal way

মডেলের জেনারাল ফর্ম

মাল্টিপল লিনিয়ার রিগ্রেশনের মডেল:

$$\hat{y} = w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b$$

ওয়েট ($w_1...w_4$): একটি ভেক্টর $W = [w_1, w_2, w_3, w_4]$
ইনপুট ফিচার ($x_1...x_4$): ফিচার ভেক্টর $X = [x_1, x_2, x_3, x_4]$

🎯 মডেল = ডট প্রোডাক্ট $(W \cdot X) + b$

"ফিচার ভেক্টর" ও "ওয়েট ভেক্টর" — কেন ভয় না পাওয়া উচিত?

অনেক সময় বড় বড় শব্দ শুনে ভয় লাগে (যেমন: ফিচার ভেক্টর, প্যারামিটার, ওয়েট, বায়াস)।

আসলে এগুলো খুবই সোজা:

টার্ম	সহজ ভাষায়
ফিচার ভেক্টর	শুধু ইনপুট ডেটাগুলোকে একসাথে রাখা (একটা লিস্ট)
ওয়েট ভেক্টর	প্রতিটি ফিচারের গুরুত্ব বোঝানোর সংখ্যা
বায়াস	একটা বাড়তি সংখ্যা যা প্রেডিকশনকে শিফট করে
প্যারামিটার	ওয়েট + বায়াস — এগুলোই মডেল শেখে

📌 মডেল যাই হোক না কেন (সিম্পল রিগ্রেশন বা চ্যাটজিপিটি) — ভিতরে কিন্তু এটাই কাজ করে: ইনপুট নেয়, ওয়েট ও বায়াস দিয়ে গুণ-যোগ করে, আউটপুট দেয়।

রিয়েল ওয়ার্ল্ড উদাহরণ — চ্যাটজিপিটি ও ডিপ লার্নিং

চ্যাটজিপিটি কী করে?

আপনি প্রশ্ন করছেন → সেটা ইনপুট
তার ভিতরে বিলিয়ন বিলিয়ন প্যারামিটার (ওয়েট + বায়াস)
প্রতিটি প্যারামিটার কিছু না কিছু গুণ-যোগ করছে
শেষে আউটপুট হিসেবে টেক্সট জেনারেট করছে

নিউরাল নেটওয়ার্কে কী থাকে?

অনেকগুলো নোড (নিউরন)
প্রতি নোডে আলাদা ওয়েট ও বায়াস

সবগুলো মিলে একটা বিশাল ম্যাথমেটিক্যাল ফাংশন তৈরি করে

🧠 মূল কথা: আপনি যতই জটিল মডেল দেখুন না কেন (LLM, Transformer, GPT) — তাদের ভিত্তি কিন্তু এই ŷ = W·X + b থেকেই শুরু। শুধু লেয়ার ও প্যারামিটার সংখ্যা বাড়ে।

কেন আমরা লুপ না চালিয়ে ভেক্টরাইজেশন করব?

লুপ পদ্ধতি (ধীর):

```
result = 0
for i in range(4):
    result += w[i] * x[i]
```

৪টি ফিচারের জন্য ৪ বার লুপ ঘুরবে
প্রতিবার একটা করে গুণ-যোগ হবে
সময় লাগবে = ৪ সেকেন্ড (যদি প্রতি অপারেশন ১ সেকেন্ড ধরি)

সমস্যা:

যদি ফিচার ১০০০ হয় → ১০০০ সেকেন্ড
ডিপ লার্নিং এ বিলিয়ন প্যারামিটার → লুপ দিয়ে অসম্ভব

ভেক্টরাইজেশন (প্যারালাল প্রসেসিং) কী?

ভেক্টরাইজেশন = একসাথে অনেকগুলো গুণ-যোগ করা

NumPy যেভাবে কাজ করে:

```
import numpy as np
result = np.dot(W, X) + b
```

এক লাইনেই সব কাজ শেষ।

কেন দ্রুত?

SIMD (Single Instruction Multiple Data): একটা নির্দেশ একসাথে বহু ডেটাতে কাজ করে
প্রসেসরের একাধিক কোর ব্যবহার করে প্যারালালি হিসাব করে
সময় লাগে ≈ ১ সেকেন্ড (ডেটা যতই হোক না কেন — আনুপাতিক না বেড়ে প্রায় ধ্রুবক)

ভিজ্যুয়াল উদাহরণ:

ফিচার	ওয়েট	গুণফল
x_1	w_1	w_1x_1
x_2	w_2	w_2x_2
x_3	w_3	w_3x_3
x_4	w_4	w_4x_4

লুপ পদ্ধতি: একের পর এক হিসাব

ভেক্টরাইজেশন: সবগুলো গুণ একসাথে → তারপর সব যোগ একসাথে

NumPy ডট প্রোডাক্ট কেন শিখেছিলাম?

আগে যখন ডট প্রোডাক্ট পড়েছিলেন, হয়তো বুঝতে পারেননি কেন দরকার
এখন স্পষ্ট:ডট প্রোডাক্ট = ওয়েট ভেক্টর আর ফিচার ভেক্টরের মধ্যে দ্রুত গুণ-যোগ করার উপায়
ম্যাট্রিক্স মাল্টিপ্লিকেশন ও একই কারণে — অনেক ডেটা একসাথে প্রসেস করতে

Implementation of Prediction function

প্রেডিকশন — কী করতে চাই?

আমাদের কাছে:

m = ডেটা পয়েন্ট সংখ্যা (এখানে ৫টি)
n = ফিচার সংখ্যা (এখানে ৪টি)
X = m×n ম্যাট্রিক্স (সব ফিচার ভেক্টর)
W = ওয়েট ভেক্টর (দৈর্ঘ্য n)
b = বায়াস (একটি স্কেলার)

লক্ষ্য: প্রতিটি ডেটা পয়েন্টের জন্য $\hat{y} = W \cdot X[i] + b$ বের করে একটি লিস্ট/অ্যারে রিটার্ন করা।

🎯 আউটপুট হবে m সংখ্যক প্রেডিকশনের একটি অ্যারে।

একটি ডেটা পয়েন্টের প্রেডিকশন হাতে-কলমে

ধরা যাক:

W = [1, 1, 1, 1] (সব ওয়েট ১)
b = 1
প্রথম ডেটা পয়েন্টের ফিচার ভেক্টর X[0] = [1, 1.5, 3.8, 1] (ধরে নিচ্ছি)

তাহলে:

```
1ŷ = (1×1) + (1×1.5) + (1×3.8) + (1×1) + 1
```

```
2    = 1 + 1.5 + 3.8 + 1 + 1
3    = 8.3
```

গুরুত্বপূর্ণ:
প্রত্যাশিত স্যালারি (y) যদি 45 হয়, তাহলে প্রেডিকশন 8.3 → ভুল অনেক বেশি।
এটাই স্বাভাবিক, কারণ ওয়েট র‍্যান্ডমলি নেওয়া হয়েছে। মডেল শিখবে কীভাবে এই ভুল কমানো যায়।

কোড ইমপ্লিমেন্টেশন (লুপ পদ্ধতি)

```
1def make_prediction(X, W, b):
2    m = X.shape[0]  # ডেটা পয়েন্ট সংখ্যা
3    pred_list = np.zeros((m,))  # ফলাফল রাখার জায়গা
4
5    for i in range(m):
6        # প্রতিটি ফিচার ভেক্টর X[i] এর সাথে W-এর ডট প্রোডাক্ট + b
7        pred_list[i] = np.dot(W, X[i]) + b
8
9    return pred_list
```

ধাপে ধাপে ব্যাখ্যা:

লাইন	কী করে
m = X.shape[0]	কতগুলো ডেটা পয়েন্ট আছে (এখানে 5)
pred_list = np.zeros((m,))	[0, 0, 0, 0, 0] তৈরি করে
for i in range(m)	i = 0, 1, 2, 3, 4 ধরে লুপ
np.dot(W, X[i])	ওয়েট ও ফিচার ভেক্টরের ডট প্রোডাক্ট
+ b	বায়াস যোগ করে
pred_list[i] = ...	i-তম প্রেডিকশন স্টোর করে

ভেরিয়েবল সাইজ নির্ধারণ

```
m = X.shape[0]    # সারির সংখ্যা = 5
n = X.shape[1]    # কলামের সংখ্যা = 4
```

X.shape[0] → কতগুলো ডেটা পয়েন্ট (m)
X.shape[1] → কতগুলো ফিচার (n)

ইনিশিয়ালাইজেশন:

```
W_init = np.ones((n,))   # [1, 1, 1, 1]
b_init = 1.0
```

📌 ওয়েট ও বায়াসের মান র‍্যান্ডমলি নেওয়া যায়। শেখার সময় এগুলো ঠিক হবে।

পুরো প্রক্রিয়ার ফ্লো (চিত্রায়িত)

```
X = 5x4 ম্যাট্রিক্স          W = [w1, w2, w3, w4]        b
[ [1,   1.5, 3.8, 1]   →   np.dot →  w1*1 + w2*1.5 + ... + b → ŷ₁
  [2,   2.0, 3.5, 2]   →   np.dot →  ...                    → ŷ₂
  ...
]
```

ফলাফল: pred_list = [$\hat{y}_1$, $\hat{y}_2$, $\hat{y}_3$, $\hat{y}_4$, $\hat{y}_5$]

কেন লুপ ঠিক আছে, কিন্তু ভেক্টরাইজেশন ভালো?

লুপ পদ্ধতি (আমরা এখানে করেছি):

প্রতিটি ডেটা পয়েন্টের জন্য আলাদা np.dot কল হয়
m=5 হলে ৫ বার লুপ, m=১০ লাখ হলে ১০ লাখ বার → ধীর

উন্নত পদ্ধতি (ম্যাট্রিক্স মাল্টিপ্লিকেশন):

```
pred_list = np.dot(X, W) + b   # এক লাইন, লুপ ছাড়া
```

এখানে X (m×n) ম্যাট্রিক্স ও W (n×) ভেক্টরের ডট প্রোডাক্ট → সব প্রেডিকশন একসাথে
নামপাই ভেতরে প্যারালাল প্রসেসিং করে → অনেক দ্রুত

🧠উল্লেখ: ন্যাস্টেড লুপের টাইম কমপ্লেক্সিটি বেশি। নামপাই ভেক্টরাইজেশন ব্যবহার করলে কোড দ্রুত হয়।

Why Visualization is tougher here

প্রশ্ন — কেন ভিজুয়ালাইজেশন দেখানো হয় না?

অনেকের মনে প্রশ্ন আসতে পারে:

সিম্পল লিনিয়ার রিগ্রেশনে তো স্ক্যাটার প্লট আর রিগ্রেশন লাইন দেখানো হয়, মাল্টিপল লিনিয়ার রিগ্রেশনে কেন ভিজুয়ালাইজেশন নেই?

উত্তর:

সিম্পল লিনিয়ার রিগ্রেশনে ২টি ডাইমেনশন থাকে:
x-অক্ষ = ফিচার (যেমন এক্সপেরিয়েন্স)
y-অক্ষ = টার্গেট (স্যালারি)
মাল্টিপল লিনিয়ার রিগ্রেশনে ২টির বেশি ডাইমেনশন থাকে (n ফিচার + ১ টার্গেট)
আমরা মানুষ ৩**D**-এর বেশি ভিজুয়ালাইজ করতে পারি না

🎯 মূল কথা: ভিজুয়ালাইজেশন না দেখানোর কারণ — আমাদের মস্তিষ্ক ৩D-এর বেশি দেখতে সক্ষম নয়, কিন্তু মডেল যেকোনো ডাইমেনশনে কাজ করতে পারে।

বিভিন্ন ডাইমেনশনে জ্যামিতিক ব্যাখ্যা

ডাইমেনশন	ফিচারের সংখ্যা	ভিজুয়াল রূপ	মডেলের আকৃতি
2D	১টি ফিচার	সমতলে বিন্দু (x, y)	**লাইন** ($y = wx + b$)
3D	২টি ফিচার	স্পেসে বিন্দু (x_1, x_2, y)	**প্লেন (তল)** ($y = w_1x_1 + w_2x_2 + b$)
4D+	৩+ ফিচার	ভিজুয়ালাইজ করা সম্ভব নয়	**হাইপারপ্লেন** (গাণিতিকভাবে সম্ভব)

উদাহরণ:

2D: এক্সপেরিয়েন্স (x) আর স্যালারি (y) → একটা লাইন ফিট করি
3D: এক্সপেরিয়েন্স (x_1) + পিএস স্কিল (x_2) আর স্যালারি (y) → একটা প্লেন (তল) ফিট করি
4D: এক্সপেরিয়েন্স + পিএস স্কিল + পস টাইম + জিপিএ → ভিজুয়ালি দেখা যায় না, তবে গাণিতিকভাবে $\hat{y} = W \cdot X + b$ কাজ করে

3D ভিজুয়ালাইজেশন (যতটা সম্ভব)

3D ডেটার জন্য তিনটি অক্ষ:

X-অক্ষ: ইয়ার্স অফ এক্সপেরিয়েন্স
Y-অক্ষ: পিএস স্কিল
Z-অক্ষ: স্যালারি

প্রতিটি ডেটা পয়েন্ট (x_1, x_2, y) স্পেসের একটি বিন্দু।

মডেল ফিটিং:

এই বিন্দুগুলোর মাঝখানে একটি প্লেন (তল) বসাই, যেন প্লেন থেকে বিন্দুগুলোর দূরত্বের বর্গের যোগফল সর্বনিম্ন হয়।

📌 প্লেনের সমীকরণ: $y = w_1x_1 + w_2x_2 + b$

3D প্লটে:

ডেটা পয়েন্টগুলো 3D স্পেসে ছড়িয়ে আছে
একটি সমতল (তল) তাদের মাঝখান দিয়ে গেছে
এই প্লেনই আমাদের প্রেডিকশন মডেল

4D ও তার বেশি ডাইমেনশন — হাইপারপ্লেন

যখন ফিচার সংখ্যা ≥ ৩, তখন আমরা আর প্লেন বা তল বলি না — বলি হাইপারপ্লেন।

হাইপারপ্লেন কী?

এটি একটি n-ডাইমেনশনাল স্পেসের (n-1) ডাইমেনশনের সমতল
উদাহরণ: 3D-তে প্লেন (2D সমতল), 4D-তে হাইপারপ্লেন (3D সমতল)

গাণিতিক রূপ:

$$\hat{y} = w_1x_1 + w_2x_2 + \ldots + w_n x_n + b$$

এটিই সব ডাইমেনশনের জন্য একই সমীকরণ।

শুধু x-এর সংখ্যা বাড়ে, কিন্তু ফর্মুলা বদলায় না।

🧠 লিনিয়ার অ্যালজেব্রার শক্তি: 2D-তে লাইন ফিট করতে পারলে, একই নিয়মে 100D-তেও ফিট করা যায় — ভিজুয়াল না দেখলেও।

কেন ভিজুয়ালাইজেশন এত বেশি জরুরি না?

আমাদের পৃথিবী **3D-**তে সীমাবদ্ধ: 4D বা তার বেশি আমরা চোখে দেখতে পারি না
গাণিতিক ধারণা পরিষ্কার থাকলেই চলে: $\hat{y} = W \cdot X + b$ বুঝলেই মডেল শেখা যায়
ভিজুয়ালাইজেশন শুধু শিক্ষার শুরুতে সাহায্য করে: একবার কনসেপ্ট বুঝলে, nD নিয়েও কাজ করতে কোনো সমস্যা নেই

✅ 2D (লাইন) ও 3D (প্লেন) বোঝাই যথেষ্ট। তার উপরে ভিজুয়াল দেখার চেষ্টা না করে গাণিতিক মডেল নিয়ে চিন্তা করা ভালো।

Cost function formulization and implementation

কেন আমরা এমএসই (MSE) ব্যবহার করব?

MSE = Mean Squared Error (মিন স্কয়ারড এরর**)**

কারণ:

এটি কনভেক্স ফাংশন — মানে গ্রাফটি একটি বোলের (U-shaped) মতো
কনভেক্স ফাংশনে গ্লোবাল মিনিমা থাকে (সবচেয়ে নিচু বিন্দু)
গ্রেডিয়েন্ট ডিসেন্ট দিয়ে এই মিনিমায় পৌঁছানো সহজ

🎯 কনভেক্স বলতে বোঝায়: ফাংশনে মাত্র একটি ন্যূনতম বিন্দু আছে — সেটাই সবচেয়ে ভালো সমাধান।

এরর বা রেসিডুয়াল কী?

প্রত্যেক ডেটা পয়েন্টের জন্য:

```
error (রেসিডুয়াল) = y_predicted - y_actual
```

আমাদের উদাহরণ (W = [1,1,1,1], b = 1):

ডেটা পয়েন্ট	y_actual (স্যালারি)	y_predicted	error = y_pred - y_actual
1	45	11.3	-33.7
2	60	14.1	-45.9
3	72	16.7	-55.3
4	85	18.8	-66.2
5	95	20.0	-75.0

📌 নেগেটিভ এরর মানে: মডেল কম প্রেডিক্ট করছে (আন্ডারপ্রেডিকশন)

কস্ট ফাংশনের ফর্মুলা (MSE)

গাণিতিক সংকেতে:

$$J(W, b) = (1/(2m)) \times \Sigma (\hat{y}_i - y_i)^2$$

প্রতিটি ডেটা পয়েন্টের জন্য (prediction - actual) বের করো
সেই মানকে স্কয়ার করো (যাতে নেগেটিভ ভ্যালু চলে যায়)
সব স্কয়ারড এরর যোগ করো (Σ)
তারপর (2m) দিয়ে ভাগ করো

🧠 **m** = ডেটা পয়েন্ট সংখ্যা (এখানে m=5)

কেন 2m দিয়ে ভাগ করি?

শুধু m দিয়ে ভাগ করলেই গড় পেতাম
কিন্তু **2m** ব্যবহার করা হয় গাণিতিক সুবিধার জন্য
পরে গ্রেডিয়েন্ট ডিসেন্টে ডেরিভেটিভ নিলে, 2 ক্যানসেল হয়ে যায় — সমীকরণ সুন্দর হয়
এটি শুধু কনভেনশন (মডেলের তুলনায় কোনো প্রভাব ফেলে না)

কোড ইমপ্লিমেন্টেশন

```
def compute_cost(X, y, W, b):
```

```
    m = X.shape[0]  # ডেটা পয়েন্ট সংখ্যা
    pred_list = make_prediction(X, W, b)  # সব প্রেডিকশন একসাথে

    error = pred_list - y          # ŷ - y
    error_squared = error ** 2     # (ŷ - y)²
    cost = np.sum(error_squared)   # Σ(ŷ - y)²
    cost = cost / (2 * m)          # ভাগ 2m দিয়ে

    return cost
```

প্রতিটি লাইনের ব্যাখ্যা:

লাইন	কাজ
m = X.shape[0]	৫টি ডেটা পয়েন্ট আছে
pred_list = make_prediction(...)	[11.3, 14.1, 16.7, 18.8, 20.0]
error = pred_list - y	[11.3-45, 14.1-60, ...] = [-33.7, -45.9, ...]
error_squared = error ** 2	[1135.69, 2106.81, ...]
cost = np.sum(error_squared)	সব যোগফল (যেমন 5000)
cost = cost / (2*m)	5000 / (2×5) = 500

আমাদের উদাহরণে কস্টের মান

W_init = [1, 1, 1, 1], b_init = 1

```
pred_list = [11.3, 14.1, 16.7, 18.8, 20.0]
y = [45, 60, 72, 85, 95]

error = [-33.7, -45.9, -55.3, -66.2, -75.0]
error_squared = [1135.69, 2106.81, 3058.09, 4382.44, 5625.00]
sum = ~16308
cost = 16308 / (2×5) = 16308 / 10 = 1630.8 ≈ 1600
```

⚠️ প্রায় **160** দেখানো হয়েছে (হয়তো ভিন্ন ডেটার জন্য)। ধারণাটি গুরুত্বপূর্ণ।

কস্ট ফাংশন কনভেক্স কেন?

MSE-এর গ্রাফ সবসময় **U-shaped** হয়
মানে একটি গ্লোবাল মিনিমা আছে — নিচের দিকে একটাই বিন্দু যেখানে কস্ট সবচেয়ে কম
গ্রেডিয়েন্ট ডিসেন্ট সেই মিনিমায় পৌঁছানোর নিশ্চয়তা দেয় (যদি লার্নিং রেট ঠিক থাকে)

Calculating Gradient

গ্রেডিয়েন্ট ডিসেন্ট — কেন ও কীভাবে?

লক্ষ্য: কস্ট ফাংশন J(W,b) কে মিনিমাইজ করা।

পদ্ধতি: প্রতিটি ওয়েট ও বায়াসকে নিচের নিয়মে আপডেট করা:

```
1w_new = w_old - α × (∂J/∂w)
```

```
2b_new = b_old - α × (∂J/∂b)
```

এখানে:

α (আলফা**) =** লার্নিং রেট (কত বড় ধাপে এগোবো)
∂J/∂w = গ্রেডিয়েন্ট (ঢাল) — বলে দেয় w-এর মান বাড়াবো না কমাবো

🧠 ঢাল পজিটিভ মানে: w কমাও (নিচে নামার জন্য)

ঢাল নেগেটিভ মানে: w বাড়াও

কস্ট ফাংশনের ডেরিভেটিভ (গ্রেডিয়েন্ট)

ওয়েটের জন্য (∂J/∂w□):

```
1∂J/∂w  = (1/m) × Σ (ŷ⁽ⁱ⁾ − y⁽ⁱ⁾) × x ⁽ⁱ⁾
```

বায়াসের জন্য (∂J/∂b):

```
1∂J/∂b = (1/m) × Σ (ŷ⁽ⁱ⁾ − y⁽ⁱ⁾)
```

⚠️ লক্ষ্য করুন: ওয়েটের ক্ষেত্রে x□⁽ⁱ⁾ গুণ হয়, বায়াসের ক্ষেত্রে শুধু এরর যোগ হয়।

সুপারস্ক্রিপ্ট ও সাবস্ক্রিপ্ট নোটেশন (ভয়ের কিছু নেই)

উপরে অনেক চিহ্ন দেখানো হয়েছে — এগুলো বোঝার জন্য নিচের নিয়ম মনে রাখলেই হবে:

চিহ্ন	অর্থ	উদাহরণ
$x^{(i)}$	i-তম ডেটা পয়েন্টের পুরো ফিচার ভেক্টর	[1.5, 3.8, 1, 2]
$x_j^{(i)}$	i-তম ডেটা পয়েন্টের j-তম ফিচার	$x_2^{(1)}$ = প্রথম ডেটার ২য় ফিচার = 3.8
$\hat{y}^{(i)}$	i-তম ডেটা পয়েন্টের প্রেডিকশন	11.3
$y^{(i)}$	i-তম ডেটা পয়েন্টের আসল মান	45

উদাহরণ (আমাদের ডেটা):

$\mathbf{x}^{(1)}$ = [1, 1.5, 3.8, 1] (প্রথম ডেটা পয়েন্ট)
$\mathbf{x}_3^{(1)}$ = 3.8 (প্রথম ডেটার ৩য় ফিচার — জিপিএ)
$\mathbf{x}_2^{(4)}$ = ২.০ (চতুর্থ ডেটার ২য় ফিচার — পিএস স্কিল)

🎯 সুপারস্ক্রিপ্ট = ডেটা পয়েন্ট (সারি)
সাবস্ক্রিপ্ট = ফিচার নম্বর (কলাম)

গ্রেডিয়েন্ট ক্যালকুলেশনের ধাপে ধাপে ব্যাখ্যা

ধাপ ১: একটি ডেটা পয়েন্টের জন্য এরর বের করি

```
error = ŷ - y
```

যদি ŷ=11.3, y=45 → error = -33.7

ধাপ ২: এই error-কে সব ওয়েটের জন্য যোগ করি

w_1 (এক্সপেরিয়েন্স) এর জন্য: error × x_1 (যেমন 1.0)
w_2 (পিএস স্কিল) এর জন্য: error × x_2 (যেমন 1.5)
w_3 (পস টাইম) এর জন্য: error × x_3 (যেমন 3.8)
w_4 (জিপিএ) এর জন্য: error × x_4 (যেমন 1.0)

ধাপ ৩: সব ডেটা পয়েন্টের জন্য একই কাজ করে সামেশন করি

ধাপ ৪: শেষে m দিয়ে ভাগ করি

সম্পূর্ণ কোড ইমপ্লিমেন্টেশন

```
def calculate_gradient(X, y, W, b):
    m = X.shape[0]    # ডেটা পয়েন্ট সংখ্যা = 5


    n = X.shape[1]    # ফিচার সংখ্যা = 4

```

```
    dj_dw = np.zeros((n,))   # ৪টি ওয়েটের জন্য ৪টি গ্রেডিয়েন্ট
    dj_db = 0.0              # বায়াসের জন্য ১টি গ্রেডিয়েন্ট

    for i in range(m):       # প্রতিটি ডেটা পয়েন্টের জন্য
        prediction = np.dot(W, X[i]) + b   # ŷ⁽ⁱ⁾
         error = prediction - y[i]          # ŷ⁽ⁱ⁾ - y⁽ⁱ⁾
         dj_db = dj_db + error              # বায়াসের সামেশন

         for j in range(n):   # প্রতিটি ফিচারের জন্য
             dj_dw[j] = dj_dw[j] + (error * X[i, j])   # error × x ⁽ⁱ⁾
             #                       ^^^^^^
             #                       i-তম ডেটার j-তম ফিচার

     # গড় বের করতে m দিয়ে ভাগ
     return dj_dw / m, dj_db / m
```

কোডের ভিজুয়াল ব্যাখ্যা:

```
i=0 (প্রথম ডেটা): error = -33.7
    j=0: dj_dw[0] += -33.7 × 1.0
    j=1: dj_dw[1] += -33.7 × 1.5
    j=2: dj_dw[2] += -33.7 × 3.8
    j=3: dj_dw[3] += -33.7 × 1.0
    dj_db += -33.7

i=1 (দ্বিতীয় ডেটা): error = -45.9
    j=0: dj_dw[0] += -45.9 × 2.0
     ... এবং এভাবে সব ডেটার জন্য
```

বায়াস গ্রেডিয়েন্ট কেন আলাদা?

> বায়াসের ডেরিভেটিভে **x**□ গুণ হয় না
> কারণ বায়াস কোন নির্দিষ্ট ফিচারের সাথে গুণ হয় না — এটি একটি ধ্রুবক যা সব প্রেডিকশনে যোগ হয়
> তাই শুধু errorএর সামেশন করলেই হয়

```
dj_db = Σ (ŷ⁽ⁱ⁾ - y⁽ⁱ⁾)
```

ইনিশিয়ালাইজেশন ও ফাংশন কল

```
W_init = np.ones((n,))   # [1, 1, 1, 1]
b_init = 1.0

dj_dw, dj_db = calculate_gradient(X, y, W_init, b_init)
```

ফলাফল (উদাহরণস্বরূপ):

```
dj_dw = [-33.7 -45.9 -55.3 -66.2] / 5  (প্রায়)
dj_db = [-33.7 -45.9 -55.3 -66.2 -75.0] / 5
```

📌 এই গ্রেডিয়েন্টগুলি বলে দেবে — W ও b-কে কোন দিকে পরিবর্তন করলে কস্ট কমবে।

Cost Optimization using Gradient Descent

গ্রেডিয়েন্ট ডিসেন্ট — কেন ও কীভাবে কাজ করে?

মূল ধারণা:

কস্ট ফাংশন J(W,b) একটি বোলের মতো আকৃতির (convex)। আমরা বোলের সবচেয়ে নিচু বিন্দু (minima) খুঁজি।

প্রক্রিয়া:

1. এলোমেলোভাবে W ও b-এর মান ধরি (প্রায়ই শূন্য দিয়ে শুরু)
2. প্রতিটি ধাপে গ্রেডিয়েন্ট (ঢাল) বের করি
3. গ্রেডিয়েন্টের বিপরীত দিকে একটু করে এগোই
4. ধাপ ২-৩ বারবার করি যতক্ষণ কস্ট না কমে

🎯 নিয়ম:

- ঢাল পজিটিভ → w কমাও
- ঢাল নেগেটিভ → w বাড়াও
- আলফা (α) ঠিক করে কত বড় ধাপ নেবে

গ্রেডিয়েন্ট ডিসেন্ট ফাংশনের কোড

```
def gradient_descent(X, y, w_input, b_input, max_iter, alpha=0.01):
    w = w_input           # ইনিশিয়াল ওয়েট ভেক্টর
    b = b_input           # ইনিশিয়াল বায়াস
    cost_memo = []        # কস্টের ইতিহাস রাখবো
    iteration = []        # ইটারেশন নম্বর রাখবো

```

```
    for i in range(max_iter):
        # 1. গ্রেডিয়েন্ট বের করি (ঢাল)
        dj_dw, dj_db = calculate_gradient(X, y, w, b)

        # 2. প্যারামিটার আপডেট করি
        w = w - alpha * dj_dw   # সব ওয়েট একসাথে আপডেট
        b = b - alpha * dj_db   # বায়াস আপডেট

        # 3. অগ্রগতি ট্র্যাক করি
        cost = compute_cost(X, y, w, b)
        cost_memo.append(cost)
        iteration.append(i)

        # প্রতি ১০০ ইটারেশনে স্ট্যাটাস প্রিন্ট করি
        if i % 100 == 0:
            print(f"Iteration {i}: Cost {cost:.4f}, dj_dw: {dj_dw}, dj_db: {dj_db:.4f}")

    return w, b, cost_memo, iteration
```

ট্রেনিং প্রক্রিয়া — সঠিক ওয়েট খোঁজা

ধাপ ১: ইনিশিয়ালাইজেশন

```
W_init = np.zeros((n,))   # [0, 0, 0, 0] — ৪টি ফিচারের জন্য
b_init = 0.0
```

ধাপ ২: গ্রেডিয়েন্ট ডিসেন্ট চালানো

```
w_final, b_final, cost_memo, iter_list = gradient_descent(
    X, y,
    w_input=W_init,
    b_input=b_init,
    max_iter=100000,       # ১ লাখ ইটারেশন
    alpha=0.01
)
```

ধাপ ৩: ফলাফল দেখা

```
print(w_final)   # প্রায় [5.09, 9.98, -2.01, 0.53]
print(b_final)   # প্রায় 0.68
```

কস্ট কীভাবে কমে? (অগ্রগতি পর্যবেক্ষণ)

ইটারেশন	কস্ট (প্রায়)	মন্তব্য
০	~188	শুরুতে অনেক ভুল
১০০	~45	দ্রুত কমছে
১০০০	~12	ধীরে কমছে
১০০০০	~2.5	প্রায় ভালো
১০০০০০	~2.3	কনভার্জড

📌 প্রথম দিকে গ্রেডিয়েন্ট বড় হয় → বড় ধাপে নেমে যায়
শেষ দিকে গ্রেডিয়েন্ট ছোট হয় → ছোট ধাপে ফাইন-টিউন হয়

ওয়েটের ব্যাখ্যা — কোন ফিচার কতটা গুরুত্বপূর্ণ?

আমরা যে ট্রু ওয়েট দিয়েছিলাম (ডেটা জেনারেট করতে) vs মডেল যা শিখেছে:

ফিচার	ট্রু ওয়েট	মডেলের ওয়েট	ব্যাখ্যা
এক্সপেরিয়েন্স	5.0	~5.09	খুব কাছাকাছি — মডেল শিখেছে
পিএস স্কিল	10.0	~9.98	প্রায় সঠিক — সবচেয়ে গুরুত্বপূর্ণ
পস টাইম	-2.0	~-2.01	নেগেটিভ — ইন্টারভিউতে খারাপ করলে স্যালারি কমে
জিপিএ	0.5	~0.53	কম ইমপ্যাক্ট — সামান্য ইতিবাচক

গুরুত্বের ক্রম (ওয়েটের মান অনুযায়ী):

পিএস স্কিল (১০) → সবচেয়ে গুরুত্বপূর্ণ
এক্সপেরিয়েন্স (৫) → মাঝারি গুরুত্ব
জিপিএ (০.৫) → কম গুরুত্ব
পস টাইম (-২) → নেতিবাচক প্রভাব (যত বেশি, তত খারাপ)

🧠 "ওয়েট বেশি মানে ফিচারটা বেশি গুরুত্বপূর্ণ" — পিএস স্কিলের ওয়েট সবচেয়ে বেশি, তাই প্রেডিকশনে এর প্রভাব সবচেয়ে বেশি।

বায়াসের ব্যাখ্যা — যখন সব ফিচার শূন্য

আমরা ভেবেছিলাম: b = 2.0

মডেল শিখেছে: b ≈ 0.68

বায়াস কী বোঝায়?

যদি কারোর এক্সপেরিয়েন্স=০, পিএস স্কিল=০, পস টাইম=০, জিপিএ=০, তবুও সে প্রায় ০.৬৮ × ১০০০ = ৬৮০ টাকা পাবে (যেমন ইন্টার্নশিপ স্টাইপেন্ড বা রেজিস্ট্রেশন ফি)।

রিয়েল ওয়ার্ল্ড উদাহরণ:

> বাস ভাড়া: প্রথম কিলোমিটারের জন্য মিনিমাম চার্জ (বায়াস) + প্রতি কিলোমিটারে বাড়তি ভাড়া (ওয়েট)
> হোটেল বুকিং: রেজিস্ট্রেশন ফি (বায়াস) + প্রতি রাতের চার্জ (ওয়েট)

নতুন ডেটার জন্য প্রেডিকশন

ধরা যাক, নতুন একজন প্রার্থী:

> এক্সপেরিয়েন্স = ২ বছর
> পিএস স্কিল = ১০
> পস টাইম = ১.৫ (মানে কিছুটা খারাপ)
> জিপিএ = ৩.৮

```
X_test = [2, 10, 1.5, 3.8]

prediction = (w_final[0] * X_test[0] +    # এক্সপেরিয়েন্স
              w_final[1] * X_test[1] +    # পিএস স্কিল
              w_final[2] * X_test[2] +    # পস টাইম
              w_final[3] * X_test[3] +    # জিপিএ
              b_final)                    # বায়াস

print(prediction)  # প্রায় 111 (মানে ১,১১,০০০ টাকা)
```

হাতে-কলমে হিসাব (প্রায়):

```
= 5.09×2 + 9.98×10 + (-2.01)×1.5 + 0.53×3.8 + 0.68
= 10.18 + 99.8 - 3.015 + 2.014 + 0.68
≈ 109.66 ≈ 110 (প্রায় ১,১০,০০০ টাকা)
```

সম্পূর্ণ ফ্লোচার্ট (শেখার যাত্রা)

```
শুরু → W=[0,0,0,0], b=0
```

```
2   ↓
3প্রত্যেক ইটারেশনে:
4   ১. গ্রেডিয়েন্ট বের করি (dj_dw, dj_db)
5   ২. W = W - α × dj_dw
6   ৩. b = b - α × dj_db
7   ৪. কস্ট হিসাব করি
8   ↓
9কস্ট কি কমছে?
10   ↓ হ্যাঁ                    ↓ না (কনভার্জড)
11চালিয়ে যাই                 থামি
12   ↓
13শেষ → W_final, b_final
14   ↓
15নতুন ডেটার জন্য প্রেডিকশন: ŷ = W·X_new + b
```

Implementation through scikit learn

সাইকিট-লার্ন — কেন ও কীভাবে?

আমরা ম্যানুয়ালি যা করেছি:

কস্ট ফাংশন লেখা
গ্রেডিয়েন্ট ক্যালকুলেশন
গ্রেডিয়েন্ট ডিসেন্ট লুপ

সাইকিট-লার্ন এই সব কাজ এক লাইনে করে দেয়!

🎯 মূল কথা: লাইব্রেরি ব্যবহার করলে কাজ দ্রুত হয়, কিন্তু ভিতরের ম্যাথ না বুঝলে সমস্যা সমাধান কঠিন। তাই আমরা প্রথমে ম্যানুয়ালি শিখেছি, এখন লাইব্রেরি শিখব।

সাইকিট-লার্নে দুই ধরনের রিগ্রেশন

ফিচার	LinearRegression	SGDRegressor
পদ্ধতি	OLS (Ordinary Least Squares) — সরাসরি গাণিতিক সমীকরণ সমাধান	স্টোকাস্টিক গ্রেডিয়েন্ট ডিসেন্ট — ইটারেটিভ
গতি	ছোট ডেটায় দ্রুত	বড় ডেটায় দ্রুত
নির্ভুলতা	একদম নিখুঁত (ডেটা ছোট হলে)	প্রায় নিখুঁত (ইটারেশন বাড়ালে)
মেমোরি ব্যবহার	বেশি (ম্যাট্রিক্স ইনভার্স দরকার)	কম (একবারে এক ডেটা পয়েন্ট)
কখন ব্যবহার করব	ছোট থেকে মাঝারি ডেটাসেট (হাজার হাজার সারি)	বিশাল ডেটাসেট (লাখ/কোটি সারি)

লিনিয়ার রিগ্রেশন (OLS) — কোড ও ব্যাখ্যা

```python
from sklearn.linear_model import LinearRegression

# মডেল অবজেক্ট তৈরি
model1 = LinearRegression()

# ট্রেনিং (লার্নিং)
model1.fit(X, y)    # X = ফিচার ম্যাট্রিক্স, y = টার্গেট

# ফলাফল বের করা
weights = model1.coef_       # ওয়েট ভেক্টর (w₁, w₂, w₃, w₄)
bias = model1.intercept_     # বায়াস (b)

print(weights)   # [5.0, 10.0, -2.0, 0.5]
print(bias)      # 2.0
```

OLS কীভাবে কাজ করে?

একটি বদ্ধ-আকৃতির (closed-form) সমীকরণ ব্যবহার করে:

$$w = (X^T X)^{-1} X^T y$$

এতে ম্যাট্রিক্স ইনভার্স ও মাল্টিপ্লিকেশন দরকার হয়।

ডেটা বড় হলে এই অপারেশন ধীর হয়ে যায়।

✅ সুবিধা: একদম সঠিক উত্তর দেয় (কোনো ইটারেশন দরকার নেই)

SGDRegressor — স্টোকাস্টিক গ্রেডিয়েন্ট ডিসেন্ট

আমরা যা করেছি (ব্যাচ গ্রেডিয়েন্ট ডিসেন্ট)

> প্রতিটি ইটারেশনে সব ডেটা পয়েন্ট ব্যবহার করে গ্রেডিয়েন্ট বের করি
> নির্ভুল, কিন্তু ডেটা বড় হলে ধীর

SGD কী করে?

> প্রতিটি ইটারেশনে শুধু ১টি র‍্যান্ডম ডেটা পয়েন্ট (বা ছোট ব্যাচ) ব্যবহার করে
> দ্রুত, কিন্তু একটু কম নির্ভুল (ইটারেশন বাড়ালে নির্ভুলতা বাড়ে)

```python
from sklearn.linear_model import SGDRegressor

```

```
# মডেল তৈরি (পেনাল্টি বন্ধ করে দিয়েছি রেগুলারাইজেশনের জন্য)
model2 = SGDRegressor(
    penalty=None,           # রেগুলারাইজেশন বন্ধ (পরে শিখবে)
    max_iter=10000000,      # অনেক বেশি ইটারেশন
    learning_rate="constant" # লার্নিং রেট ফিক্সড রাখব
)

# ট্রেনিং
model2.fit(X, y)

# ফলাফল
weights = model2.coef_      # প্রায় [5.1, 9.8, -1.9, 0.55]
bias = model2.intercept_    # প্রায় 0.68
```

কেন SGDRegressor একদম সঠিক মান দিচ্ছে না?

আমাদের ম্যানুয়াল গ্রেডিয়েন্ট ডিসেন্ট (ব্যাচ):

সব ডেটা পয়েন্ট একসাথে নিয়ে গ্রেডিয়েন্ট বের করি
নির্ভুলভাবে ফাইভ,টেন,মাইনাসট, পই5 ও 2 পায়

SGDRegressor:

প্রতি ইটারেশনে ১টি র্যান্ডম ডেটা পয়েন্ট নিয়ে গ্রেডিয়েন্ট বের করে
ফলে কিছুটা নড়বড়ে (noisy) আপডেট হয়
অনেক ইটারেশন দিলেও exact মান না-ও আসতে পারে

কিভাবে মান কাছাকাছি আনব?

max_iter বাড়াও (যত বাড়াবে, তত কাছাকাছি যাবে)
learning_rate ঠিক করো (constant রেখে eta0 দিয়ে মান দাও)
ডেটা স্কেলিং করো (StandardScaler — পরে শিখবে)

আমাদের রান (১০,০০০ ইটারেশন):

```
1OLS: [5.0, 10.0, -2.0, 0.5], bias=2.0
2SGD: [6.2, 8.8, -0.7, 1.36], bias=0.65
```

SGD এর মান কিছুটা দূরে। কিন্তু ১০,০০০,০০ ইটারেশন দিলে আরও কাছাকাছি আসবে।

কখন কোনটি ব্যবহার করব?

ডেটা সাইজ	সুপারিশ
ছোট (হাজার হাজার সারি)	LinearRegression (OLS) — দ্রুত ও নিখুঁত
মাঝারি (লক্ষ সারি)	LinearRegression (যদি মেমোরি অনুমতি দেয়) অথবা SGDRegressor
বিশাল (কোটি সারি)	SGDRegressor — OLS অসম্ভব ধীর
অনলাইন লার্নিং (নতুন ডেটা আসতে থাকে)	SGDRegressor (partial_fit মেথড)

আমরা এত ম্যানুয়ালি কেন করলাম?

সাইকিট-লার্ন ব্যবহার করলে তো ২ লাইনেই কাজ শেষ!

কিন্তু শেখার উদ্দেশ্য ছিল:

- মডেলের অন্তর্জাতিক ম্যাথ বোঝা
- গ্রেডিয়েন্ট ডিসেন্টের লজিক পরিষ্কার হওয়া
- ওয়েট ও বায়াসের অর্থ বোঝা (কোন ফিচার কতটা গুরুত্বপূর্ণ)
- হাইপারপ্যারামিটার টিউনিং (লার্নিং রেট, ইটারেশন) বুঝা
- লাইব্রেরি ব্যবহার করলেও ভুল ডিবাগ করতে পারা

🧠 শেখার সঠিক ক্রম:

ম্যানুয়াল → লাইব্রেরি

কারণ লাইব্রেরি ব্যবহার করতে গেলেও ভিতরের ম্যাথ জানা থাকলে প্যারামিটার ঠিক করা সহজ হয়।